



МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
Кафедра математического и компьютерного моделирования

Кулахсзян Сергей Грайрович

МЕТОД BOOTSTRAP

ЛАБОРАТОРНАЯ РАБОТА № 4

Направление подготовки 02.03.01 сэт Сквозные цифровые технологии,
бакалавриатская программа «Математика и компьютерные науки»
Очной формы обучения

г. Владивосток
2026

Содержание

1	Реализация метода bootstrap	2
2	Сравнение собственного метода bootstrap с реализацией из scipy.stats	3
2.1	Равномерное распределение	3
2.2	Распределение Бернулли	6
2.3	Биномиальное распределение	8
2.4	Нормальное распределение	11
3	Табличный результат	14

1 Реализация метода bootstrap

Метод bootstrap:

Имеется выборка $X_1, \dots, X_n$ из X

α – уровень доверия

Алгоритм вычисления доверительного интервала параметра θ

1. Найти оценку параметра $\hat{\theta}_n$
2. Сгенерировать k новых выборок $X_1^i, \dots, X_n^i$, $i = \overline{1, k}$, X_j^i – случайный элемент исходной выборки
3. $\theta_i^* = \hat{\theta}_n(X_1^i, \dots, X_n^i)$, $i = \overline{1, k}$
4. Доверительный интервал – $\left(\theta_{\left\lfloor \frac{1-\alpha}{2} k \right\rfloor}^*; \theta_{\left\lceil \frac{1+\alpha}{2} k \right\rceil}^* \right)$

Моя реализация:

```
1 def bootstrap(X_n: List[float], theta_n:
  Callable[[List[float]], float], alpha: float = 0.05) ->
  Tuple[float, float]:
2     k = 1000
3     theta_i = [0] * k
4
5     for i in range(k):
6         X_i = random.choices(X_n, k=len(X_n))
7         theta_i[i] = theta_n(X_i)
8     theta_i.sort()
9
10    left = math.floor((1 - alpha) / 2 * k)
11    right = math.ceil((1 + alpha) / 2 * k)
12
13    return theta_i[left], theta_i[right]
```

2 Сравнение собственного метода bootstrap с реализацией из `scipy.stats`

2.1 Равномерное распределение

Оценим параметры равномерного распределения методом максимального правдоподобия:

$$X \sim U(a, b)$$

$X_1, \dots, X_n$ — выборка из X

$$F_X \in \{F(x, a, b) \mid a, b \in \mathbb{R}, b \geq a\}$$

$$f_X(x) = \frac{I(x \in [a; b])}{(b - a)^n}$$

$$L(a, b, X_1, \dots, X_n) = \prod_{i=1}^n \frac{I(X_i \in [a; b])}{(b - a)^n} = \frac{1}{(b - a)^n} \prod_{i=1}^n I(X_i \in [a; b])$$

$$\hat{a}_n = \arg \max_{a \leq b} \frac{1}{(b - a)^n} \prod_{i=1}^n I(X_i \in [a; b]) = \arg \max_{a \leq X_{\min}} \frac{1}{(b - a)^n} = X_{\min}$$

$$\hat{b}_n = \arg \max_{b \geq a} \frac{1}{(b - a)^n} \prod_{i=1}^n I(X_i \in [a; b]) = \arg \max_{b \geq X_{\max}} \frac{1}{(b - a)^n} = X_{\max}$$

Оценка параметра $\hat{a}_n = X_{\min}$

Оценка параметра $\hat{b}_n = X_{\max}$

Код:

```
1 def a_n_U(X_n: List[float]) -> float:
2     return min(X_n)
3 def b_n_U(X_n: List[float]) -> float:
4     return max(X_n)
5
6
7 sample_100 = stats.uniform.rvs(loc=-1, scale=5, size=100,
8     random_state=random_state)
9
10 data["my"]["uniform"][100]["a_n"] = bootstrap(
11     X_n=sample_100,
12     theta_n=a_n_U
13 )
```

```

14 data["my"]["uniform"][100]["b_n"] = bootstrap(
15     X_n=sample_100,
16     theta_n=b_n_U
17 )
18
19 res = stats.bootstrap(
20     data=(sample_100,),
21     statistic=a_n_U,
22     vectorized=False,
23     confidence_level=0.95,
24     n_resamples=1000,
25     random_state=random_state
26 )
27 data["scipy"]["uniform"][100]["a_n"] = (
28     res.confidence_interval.low,
29     res.confidence_interval.high
30 )
31 res = stats.bootstrap(
32     data=(sample_100,),
33     statistic=b_n_U,
34     vectorized=False,
35     confidence_level=0.95,
36     n_resamples=1000,
37     random_state=random_state
38 )
39 data["scipy"]["uniform"][100]["b_n"] = (
40     res.confidence_interval.low,
41     res.confidence_interval.high
42 )
43
44
45
46 sample_1000 = stats.uniform.rvs(loc=-1, scale=5, size=100,
47     random_state=random_state)
48
49 data["my"]["uniform"][1000]["a_n"] = bootstrap(
50     X_n=sample_1000,
51     theta_n=a_n_U

```

```

52 )
53 data["my"]["uniform"][1000]["b_n"] = bootstrap(
54     X_n=sample_1000,
55     theta_n=b_n_U
56 )
57
58 res = stats.bootstrap(
59     data=(sample_1000,),
60     statistic=a_n_U,
61     vectorized=False,
62     confidence_level=0.95,
63     n_resamples=1000,
64     random_state=random_state
65 )
66 data["scipy"]["uniform"][1000]["a_n"] = (
67     res.confidence_interval.low,
68     res.confidence_interval.high
69 )
70 res = stats.bootstrap(
71     data=(sample_1000,),
72     statistic=b_n_U,
73     vectorized=False,
74     confidence_level=0.95,
75     n_resamples=1000,
76     random_state=random_state
77 )
78 data["scipy"]["uniform"][1000]["b_n"] = (
79     res.confidence_interval.low,
80     res.confidence_interval.high
81 )
82
83
84
85 print(data["my"]["uniform"])
86 print(data["scipy"]["uniform"])

```

2.2 Распределение Бернулли

Оценим параметр биномиального распределения методом максимального правдоподобия:

$$X \sim \text{Bernoulli}(p) : X_1, X_2, \dots, X_n$$

$$F_X \in \{F_X(x, p) | p \in [0; 1]\}$$

$$p_X(x) = p^x (1 - p)^{1-x}$$

$$L(p, X_1, \dots, X_n) = \prod_{i=1}^n p^{X_i} (1 - p)^{1-X_i} = p^{\sum_{i=1}^n X_i} (1 - p)^{n - \sum_{i=1}^n X_i}$$

$$\ln L(p, X_1, \dots, X_n) = \ln \left(p^{\sum_{i=1}^n X_i} (1 - p)^{n - \sum_{i=1}^n X_i} \right) =$$

$$\sum_{i=1}^n X_i \ln p + \left(n - \sum_{i=1}^n X_i \right) \ln(1 - p)$$

$$\hat{p}_n = \arg \max_{p \in [0; 1]} \ln L(p, X_1, \dots, X_n)$$

Находим максимума по аргументу:

$$\frac{d \ln L}{dp} = \frac{\sum_{i=1}^n X_i}{p} + \frac{\sum_{i=1}^n X_i - n}{1 - p} = 0$$

$$(1 - p) \sum_{i=1}^n X_i + \left(\sum_{i=1}^n X_i - n \right) p = 0$$

$$pn = \sum_{i=1}^n X_i$$

$$p = \sum_{i=1}^n \frac{X_i}{n}$$

Проверка максимума:

$$\frac{d^2 \ln L}{dp^2} = \underbrace{-\frac{\sum_{i=1}^n X_i}{p^2}}_{\leq 0} + \underbrace{\frac{\sum_{i=1}^n X_i - n}{(1 - p)^2}}_{\leq 0} \leq 0$$

$$\text{Оценка параметра } \hat{p}_n = \sum_{i=1}^n \frac{X_i}{n} = \bar{X}$$

Код:

```
1 def p_n_Bernoulli(X_n: List[float]) -> float:
2     return sum(X_n) / len(X_n)
3
```

```

4
5 sample_100 = stats.bernoulli.rvs(p=0.25, size=100,
  random_state=random_state)
6
7
8 data["my"]["bernoulli"][100] = bootstrap(
9     X_n=sample_100,
10    theta_n=p_n_Bernoulli
11 )
12
13 res = stats.bootstrap(
14     data=(sample_100,),
15     statistic=p_n_Bernoulli,
16     vectorized=False,
17     confidence_level=0.95,
18     n_resamples=1000,
19     random_state=random_state
20 )
21 data["scipy"]["bernoulli"][100] = (
22     res.confidence_interval.low,
23     res.confidence_interval.high
24 )
25
26
27
28 sample_1000 = stats.bernoulli.rvs(p=0.25, size=1000,
  random_state=random_state)
29
30
31 data["my"]["bernoulli"][1000] = bootstrap(
32     X_n=sample_1000,
33     theta_n=p_n_Bernoulli
34 )
35
36 res = stats.bootstrap(
37     data=(sample_1000,),
38     statistic=p_n_Bernoulli,
39     vectorized=False,
40     confidence_level=0.95,

```



```

41     n_resamples=1000,
42     random_state=random_state
43 )
44 data["scipy"]["bernoulli"][1000] = (
45     res.confidence_interval.low,
46     res.confidence_interval.high
47 )
48
49
50
51 print(data["my"]["bernoulli"])
52 print(data["scipy"]["bernoulli"])

```

2.3 Биномиальное распределение

Оценим параметры биномиального распределения методом моментов:

$X \sim \text{Bin}(k, p)$

$X_1, \dots, X_n$ — выборка из X

$F_X \in \{F_X(x, k, p) \mid p \in [0; 1], k \in \mathbb{N}\}$

$$\begin{cases} E(X) = \bar{X} \\ D(X) = \overline{X^2} - \bar{X}^2 \end{cases} \Leftrightarrow \begin{cases} kp = \bar{X} \\ kp(1-p) = \overline{X^2} - \bar{X}^2 \end{cases} \Leftrightarrow \begin{cases} p = \frac{\bar{X} - \overline{X^2} + \bar{X}^2}{\bar{X}} \\ k = \frac{\bar{X}^2}{\bar{X} - \overline{X^2} + \bar{X}^2} \end{cases}$$

Оценка параметра $\hat{p}_n = \frac{\bar{X} - \overline{X^2} + \bar{X}^2}{\bar{X}}$

Оценка параметра $\hat{k}_n = \frac{\bar{X}^2}{\bar{X} - \overline{X^2} + \bar{X}^2}$

Код:

```

1 def p_n_Bin(X_n: List[float]) -> float:
2     X_mean = sum(X_n) / len(X_n)
3     X_2_mean = sum(X ** 2 for X in X_n) / len(X_n)
4
5     return (X_mean - X_2_mean + X_mean ** 2) / X_mean
6
7 def k_n_Bin(X_n: List[float]) -> float:
8     X_mean = sum(X_n) / len(X_n)

```

```

9     X_2_mean = sum(X ** 2 for X in X_n) / len(X_n)
10
11     return X_mean ** 2 / (X_mean - X_2_mean + X_mean ** 2)
12
13
14 sample_100 = stats.binom.rvs(p=0.25, n=17, size=100,
15 random_state=random_state)
16
17 data["my"]["binom"][100]["p_n"] = bootstrap(
18     X_n=sample_100,
19     theta_n=p_n_Bin
20 )
21 data["my"]["binom"][100]["k_n"] = bootstrap(
22     X_n=sample_100,
23     theta_n=k_n_Bin
24 )
25
26 res = stats.bootstrap(
27     data=(sample_100,),
28     statistic=p_n_Bin,
29     vectorized=False,
30     confidence_level=0.95,
31     n_resamples=1000,
32     random_state=random_state
33 )
34 data["scipy"]["binom"][100]["p_n"] = (
35     res.confidence_interval.low,
36     res.confidence_interval.high
37 )
38 res = stats.bootstrap(
39     data=(sample_100,),
40     statistic=k_n_Bin,
41     vectorized=False,
42     confidence_level=0.95,
43     n_resamples=1000,
44     random_state=random_state
45 )
46 data["scipy"]["binom"][100]["k_n"] = (

```

```

47     res.confidence_interval.low,
48     res.confidence_interval.high
49 )
50
51
52 sample_1000 = stats.binom.rvs(p=0.25, n=17, size=1000,
53                               random_state=random_state)
54
55 data["my"]["binom"][1000]["p_n"] = bootstrap(
56     X_n=sample_1000,
57     theta_n=p_n_Bin
58 )
59 data["my"]["binom"][1000]["k_n"] = bootstrap(
60     X_n=sample_1000,
61     theta_n=k_n_Bin
62 )
63
64 res = stats.bootstrap(
65     data=(sample_1000,),
66     statistic=p_n_Bin,
67     vectorized=False,
68     confidence_level=0.95,
69     n_resamples=1000,
70     random_state=random_state
71 )
72 data["scipy"]["binom"][1000]["p_n"] = (
73     res.confidence_interval.low,
74     res.confidence_interval.high
75 )
76 res = stats.bootstrap(
77     data=(sample_1000,),
78     statistic=k_n_Bin,
79     vectorized=False,
80     confidence_level=0.95,
81     n_resamples=1000,
82     random_state=random_state
83 )
84 data["scipy"]["binom"][1000]["k_n"] = (

```

```

85     res.confidence_interval.low,
86     res.confidence_interval.high
87 )
88
89
90
91 print(data["my"]["binom"])
92 print(data["scipy"]["binom"])

```

2.4 Нормальное распределение

Оценим параметры нормального распределения методом моментов:

$$X \sim N(\mu, \sigma^2)$$

$X_1, \dots, X_n$ — выборка из X

$$F_X \in \{F_X(x, \mu, \sigma^2) \mid \mu, \sigma \in \mathbb{R}\}$$

$$\begin{cases} E(X) = \bar{X} \\ D(X) = \overline{X^2} - \bar{X}^2 \end{cases} \Leftrightarrow \begin{cases} \mu = \bar{X} \\ \sigma^2 = \overline{X^2} - \bar{X}^2 \end{cases} \Leftrightarrow \begin{cases} \mu = \bar{X} \\ \sigma = \sqrt{\overline{X^2} - \bar{X}^2} \end{cases}$$

Оценка параметра $\hat{\mu}_n = \bar{X}$

Оценка параметра $\hat{\sigma}_n = \sqrt{\overline{X^2} - \bar{X}^2}$

Код:

```

1  def mu_n_N(X_n: List[float]) -> float:
2      return sum(X_n) / len(X_n)
3
4  def sigma_n_N(X_n: List[float]) -> float:
5      X_mean = sum(X_n) / len(X_n)
6      X_2_mean = sum(X ** 2 for X in X_n) / len(X_n)
7
8      return math.sqrt(X_2_mean - X_mean ** 2)
9
10
11 sample_100 = stats.norm.rvs(loc=15, scale=5, size=100,
12     random_state=random_state)
13
14 data["my"]["norm"][100]["mu_n"] = bootstrap(

```

```

15     X_n=sample_100,
16     theta_n=mu_n_N
17 )
18 data["my"]["norm"][100]["sigma_n"] = bootstrap(
19     X_n=sample_100,
20     theta_n=sigma_n_N
21 )
22
23 res = stats.bootstrap(
24     data=(sample_100,),
25     statistic=mu_n_N,
26     vectorized=False,
27     confidence_level=0.95,
28     n_resamples=1000,
29     random_state=random_state
30 )
31 data["scipy"]["norm"][100]["mu_n"] = (
32     res.confidence_interval.low,
33     res.confidence_interval.high
34 )
35 res = stats.bootstrap(
36     data=(sample_100,),
37     statistic=sigma_n_N,
38     vectorized=False,
39     confidence_level=0.95,
40     n_resamples=1000,
41     random_state=random_state
42 )
43 data["scipy"]["norm"][100]["sigma_n"] = (
44     res.confidence_interval.low,
45     res.confidence_interval.high
46 )
47
48
49 sample_1000 = stats.norm.rvs(loc=15, scale=5, size=1000,
50     random_state=random_state)
51
52 data["my"]["norm"][1000]["mu_n"] = bootstrap(

```

```

53     X_n=sample_1000,
54     theta_n=mu_n_N
55 )
56 data["my"]["norm"][1000]["sigma_n"] = bootstrap(
57     X_n=sample_1000,
58     theta_n=sigma_n_N
59 )
60
61 res = stats.bootstrap(
62     data=(sample_1000,),
63     statistic=mu_n_N,
64     vectorized=False,
65     confidence_level=0.95,
66     n_resamples=1000,
67     random_state=random_state
68 )
69 data["scipy"]["norm"][1000]["mu_n"] = (
70     res.confidence_interval.low,
71     res.confidence_interval.high
72 )
73 res = stats.bootstrap(
74     data=(sample_1000,),
75     statistic=sigma_n_N,
76     vectorized=False,
77     confidence_level=0.95,
78     n_resamples=1000,
79     random_state=random_state
80 )
81 data["scipy"]["norm"][1000]["sigma_n"] = (
82     res.confidence_interval.low,
83     res.confidence_interval.high
84 )
85
86
87
88 print(data["my"]["norm"])
89 print(data["scipy"]["norm"])

```

3 Табличный результат

Распределение	Параметр	Значение	Размер выборки	Моя	SciPy
Равномерное	a	-1.00	100	(-0.9449, -0.9449)	(-0.9449, -0.9383)
			1000	(-0.9449, -0.9449)	(-0.9449, -0.9383)
	b	4.00	100	(3.9869, 3.9869)	(3.9425, 3.9869)
			1000	(3.9869, 3.9869)	(3.9425, 3.9869)
Бернулли	p	0.25	100	(0.27, 0.27)	(0.19, 0.36)
			1000	(0.242, 0.244)	(0.2160, 0.2693)
Биномиальное	k	17.00	100	(21.2259, 22.9142)	(10.7230, 1092.8516)
			1000	(18.2573, 18.5386)	(14.6276, 28.7498)
	p	0.25	100	(0.1482, 0.1670)	(-0.1193, 0.3526)
			1000	(0.2300, 0.2347)	(0.1494, 0.2894)
Нормальное	μ	15.00	100	(15.8747, 15.9280)	(14.9171, 17.0923)
			1000	(14.9498, 14.9738)	(14.6358, 15.2777)
	σ	5.00	100	(5.4637, 5.5104)	(4.7767, 6.2535)
			1000	(5.1785, 5.1918)	(4.9801, 5.4431)